

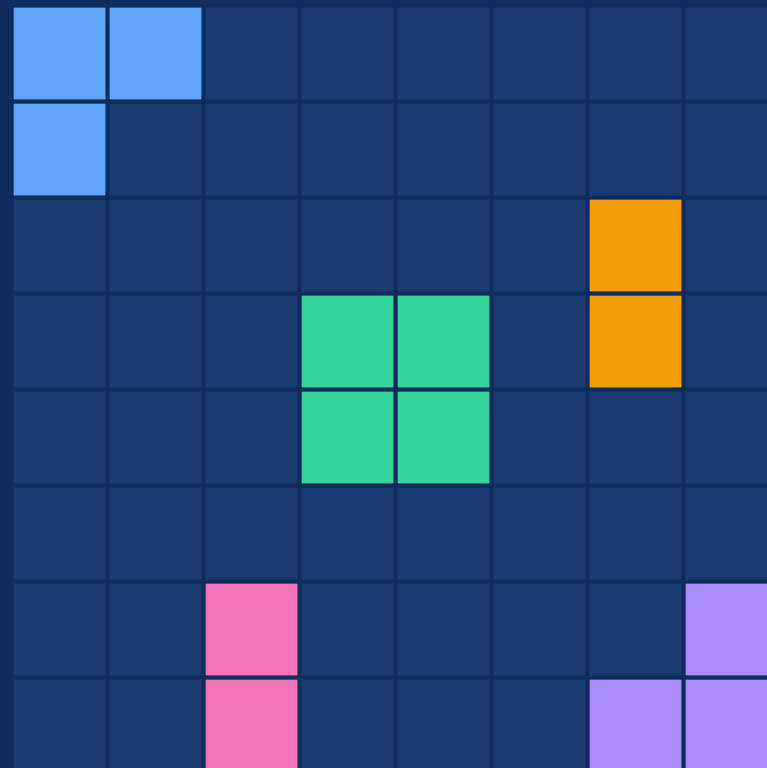
# Contagem paralela de objetos em matriz binária

Implementações sequencial e paralela com Pthreads

---

Júlia Bettiol de Oliveira · Thomaz Szeckir Gomes

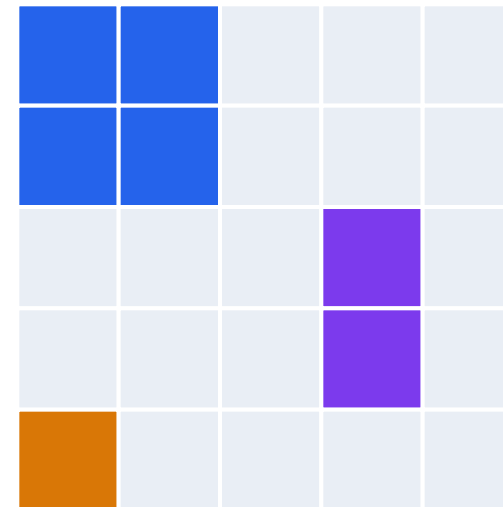
Sistemas Operacionais · PUCRS · 2026/II · Prof. Filipo Mór



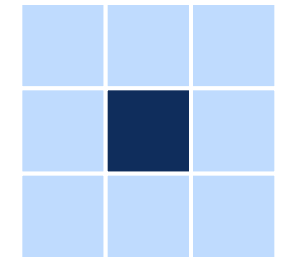
Exemplo 3 do enunciado: 5 objetos

# Contar objetos em uma matriz binária

- Matriz binária: **1** é primeiro plano, **0** é fundo.
- **Objeto**: região de 1s ligados por lados ou cantos, a **conectividade 8**.
- Saída: a **quantidade de objetos** da matriz.
- No paralelo, um objeto pode **cruzar a divisão** entre threads. Somar as contagens locais contaria o mesmo objeto duas vezes.



Exemplo 1: 3 objetos



os 8 vizinhos de uma célula

A versão sequencial é a referência de correção: a paralela precisa dar **exatamente** o mesmo número, com qualquer quantidade de threads.

# Visão geral da solução

1

## Ler ou gerar

Matriz guardada em um vetor 1D, linha a linha.

sequencial

2

## Dividir

T faixas contíguas de linhas, uma por thread.

sequencial

3

## Rotular

Cada thread faz flood fill só na sua faixa.

paralelo

4

## Unir fronteiras

Union-Find liga os pedaços de um mesmo objeto.

sequencial

5

## Contar

Objetos =  $\Sigma$  componentes locais – uniões.

sequencial

## Versão sequencial

Um único fluxo de controle. Referência de correção e de desempenho.

```
./bin/sequencial
```

## Versão paralela (Pthreads)

Quantidade de threads configurável e o mesmo resultado da sequencial.

```
./bin/paralelo <threads>
```

A etapa 3 é a parte pesada e roda em paralelo. As etapas 4 e 5 custam proporcional às fronteiras,  $(T - 1) \times C$ , e não à matriz inteira.

## Referência: flood fill iterativo

- Varre a matriz: cada **1 ainda não visitado** começa um objeto novo.
- O **flood fill** marca o objeto inteiro como visitado.
- Pilha no **heap**, sem recursão: objetos com milhões de células não estouram a pilha de execução.
- A célula é marcada **ao empilhar**, então nunca entra duas vezes na pilha.
- Vetor **visitado** com 1 byte por célula.

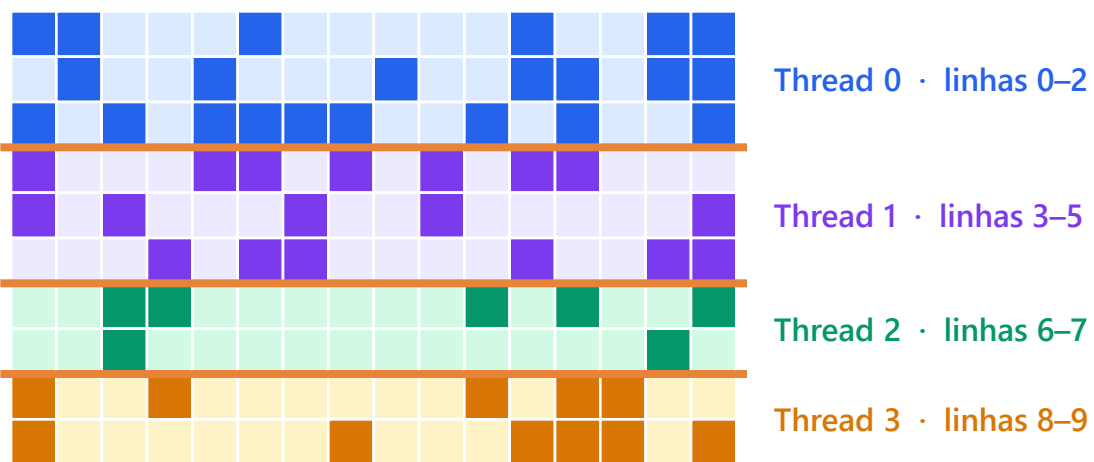
conta-objetos-sequencial.c (resumido)

```
for (i = 0; i < total; i++)
    if (m[i] == 1 && visitado[i] == 0) {
        objetos++;
        preenche_regiao(m, L, C, visitado, pilha, i);
    }

/* preenche_regiao: pilha explícita no heap */
visitado[inicio] = 1; pilha[topo++] = inicio;
while (topo > 0) {
    atual = pilha[--topo];
    for (k = 0; k < 8; k++) { /* 8 vizinhos */
        ni = i + DI[k]; nj = j + DJ[k];
        if (fora da matriz) continue;
        if (m[idx] == 1 && visitado[idx] == 0) {
            visitado[idx] = 1; pilha[topo++] = idx;
        }
    }
}
```

DI e DJ guardam os 8 deslocamentos: as 4 direções retas e as 4 diagonais.

# Matriz dividida em faixas de linhas



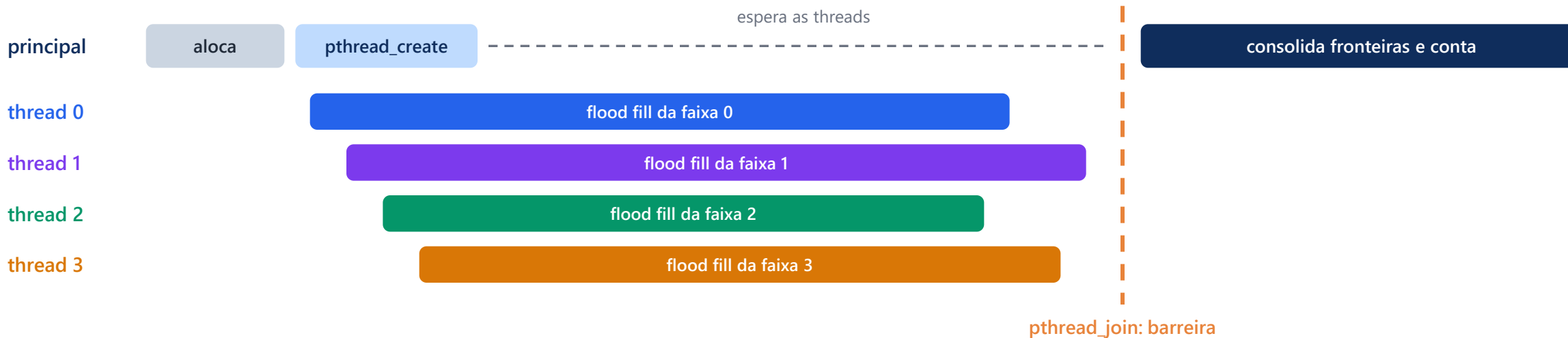
Ex.:  $L = 10$  e  $T = 4$ :  $10 / 4 = 2$ , resto 2, então faixas de 3, 3, 2 e 2 linhas.

- Número de threads na linha de comando: **`./bin/paralelo T`**.
- Cada faixa recebe  $L / T$  linhas; as primeiras  $L \% T$  faixas ganham uma linha a mais.
- Cada thread faz flood fill **só dentro da sua faixa**.
- Rótulo de um componente = **índice da 1ª célula + 1**: único na matriz, sem as threads combinarem nada.
- Com 2 threads (ex. 2 e 3) e 3 threads (ex. 4 e 5), os cortes caem nas **linhas laranja do enunciado**.

## Por que faixas de linhas?

A matriz é um vetor 1D linha a linha: cada faixa é um bloco contíguo de memória, e só existem fronteiras horizontais para consolidar.

# Uma barreira e nenhum lock



## Matriz de entrada

Somente leitura: nenhuma thread escreve nela.

## rotulos[ ] e pai[ ]

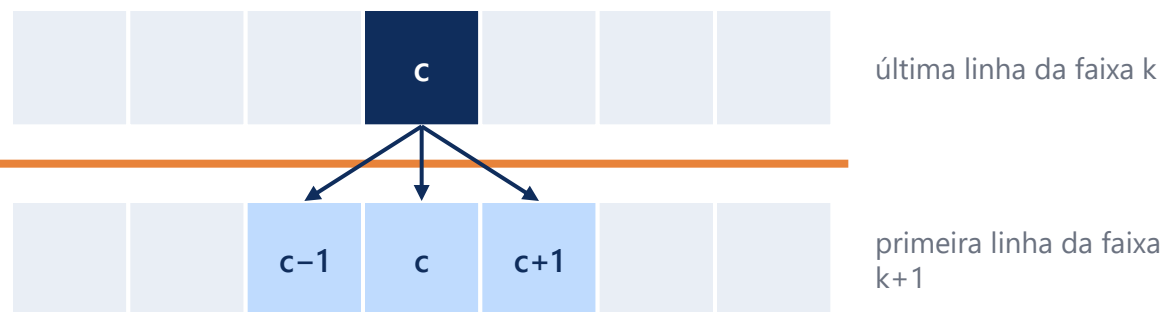
Cada thread escreve só posições da própria faixa: regiões disjuntas, sem mutex.

## Union-Find

As uniões acontecem só na thread principal, depois do join.

Sem locks não há deadlock. Valgrind **helgrind** e **drd**: 0 condições de corrida detectadas.

# Unindo objetos que cruzam as faixas



Para cada 1 da última linha: 3 comparações, a vertical e as duas diagonais.

## E o encontro de quatro blocos?

Só existe quando há cortes verticais e horizontais ao mesmo tempo. Como dividimos só por linhas, todo cruzamento, inclusive o diagonal, passa por uma fronteira horizontal e é coberto por  $c-1$  e  $c+1$ .

- Horizontal: resolvida **dentro da faixa** pelo flood fill.
- Na fronteira, rótulos diferentes dos dois lados: **uf\_union(a, b)**.
- uf\_union devolve **1 só quando junta conjuntos distintos**: a união efetiva.
- Cada componente local começa como um conjunto; cada união efetiva junta dois em um.

$$\text{objetos} = \Sigma \text{ locais} - \text{uniões efetivas}$$

sem varrer a matriz de novo depois do join

## Exemplo 2 do enunciado com 2 threads

Rótulos locais de cada thread

|   |   |   |   |   |  |   |   |
|---|---|---|---|---|--|---|---|
|   |   |   |   |   |  | A | A |
|   | B | B | B | B |  | A |   |
|   |   | B | B |   |  |   |   |
|   |   |   | C | C |  |   |   |
|   |   |   |   | C |  |   | D |
| E | E |   |   |   |  | D | D |

uf\_union(B, C)

Depois do Union-Find

|   |   |   |   |   |  |   |   |
|---|---|---|---|---|--|---|---|
|   |   |   |   |   |  | A | A |
|   | B | B | B | B |  | A |   |
|   |   | B | B |   |  |   |   |
|   |   |   | B | B |  |   |   |
|   |   |   |   | B |  |   | D |
| E | E |   |   |   |  | D | D |

A linha laranja divide a thread 0 (linhas 0–2) e a thread 1 (linhas 3–5), como no enunciado. B e C são o mesmo objeto, partido pela divisão.

Thread 0 · linhas 0–2

**2** componentes: A e B

Thread 1 · linhas 3–5

**3** componentes: C, D e E

Soma dos componentes locais

**5**

Uniões efetivas na fronteira

**1**: B com C**5 – 1 = 4 objetos**

Igual ao esperado pelo enunciado.

# Mesmo resultado em todas as configurações

| Ex. | Dimensões | Esperado | Sequencial | Paralelo |   |
|-----|-----------|----------|------------|----------|---|
| 1   | 5 × 5     | 3        | 3          | 3        | ✓ |
| 2   | 6 × 8     | 4        | 4          | 4        | ✓ |
| 3   | 8 × 8     | 5        | 5          | 5        | ✓ |
| 4   | 9 × 12    | 6        | 6          | 6        | ✓ |
| 5   | 12 × 12   | 7        | 7          | 7        | ✓ |

Paralelo testado com 2, 3, 4 e 8 threads: mesmo resultado em todas.

**Testes adicionais:** só zeros: 0 · objeto diagonal 8 × 8: 1 · diagonal 6 × 6: 1

625

comparações sequencial × paralelo  
em matrizes aleatórias, de 2 a 16  
threads: 0 divergências

0

condições de corrida no Valgrind  
helgrind e drd

0

vazamentos de memória no Valgrind  
memcheck

0

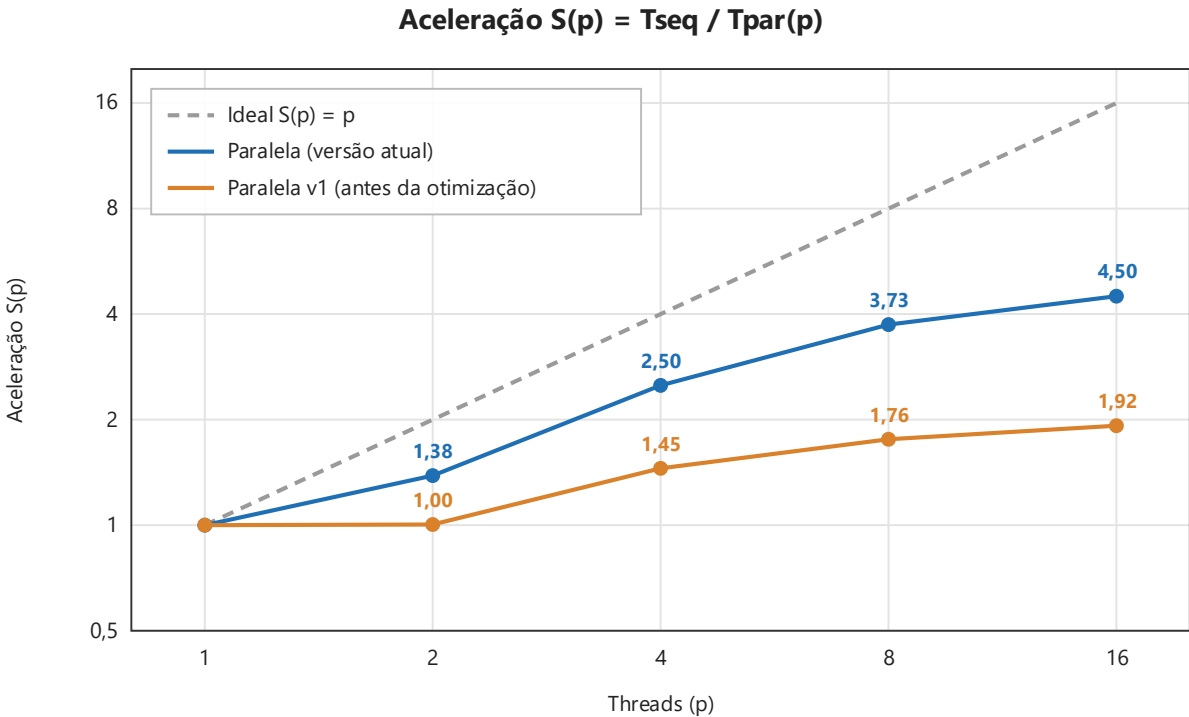
aviso do compilador em modo C89  
estrito, com todos os alertas ligados

# Matriz 4000 × 4000

| Threads    | Tempo (ms) | S    | E    |
|------------|------------|------|------|
| sequencial | 234,6      | 1,00 | 1,00 |
| 2          | 169,4      | 1,38 | 0,69 |
| 4          | 93,7       | 2,50 | 0,63 |
| 8          | 62,8       | 3,73 | 0,47 |
| 16         | 52,1       | 4,50 | 0,28 |

**S** = Tsequencial / Tparalelo      **E** = S / p

Mediana de 5 repetições intercaladas sobre a mesma matriz: 16 milhões de células, densidade 35%, semente 2026. Intel i7-14700HX, Ubuntu no WSL2, gcc -O2.



Ganho real, mas abaixo do ideal: a memória vira gargalo, o i7 mistura núcleos rápidos e lentos e uma parte continua sequencial.

# Primeira versão × versão atual

| Threads | v1 (ms) | atual (ms) | ganho |
|---------|---------|------------|-------|
| 2       | 233,5   | 169,4      | 1,38× |
| 4       | 161,4   | 93,7       | 1,72× |
| 8       | 133,5   | 62,8       | 2,13× |
| 16      | 122,1   | 52,1       | 2,34× |

- A v1 percorria a **matriz inteira** duas vezes na thread principal: para inicializar pai[ ] e para recontar as raízes.
- Essa parte não diminui com mais threads: **Lei de Amdahl**. Aceleração máxima da v1: **1,92**.
- A versão atual elimina as duas passadas e chega a **4,50**.

Antes (v1), na thread principal

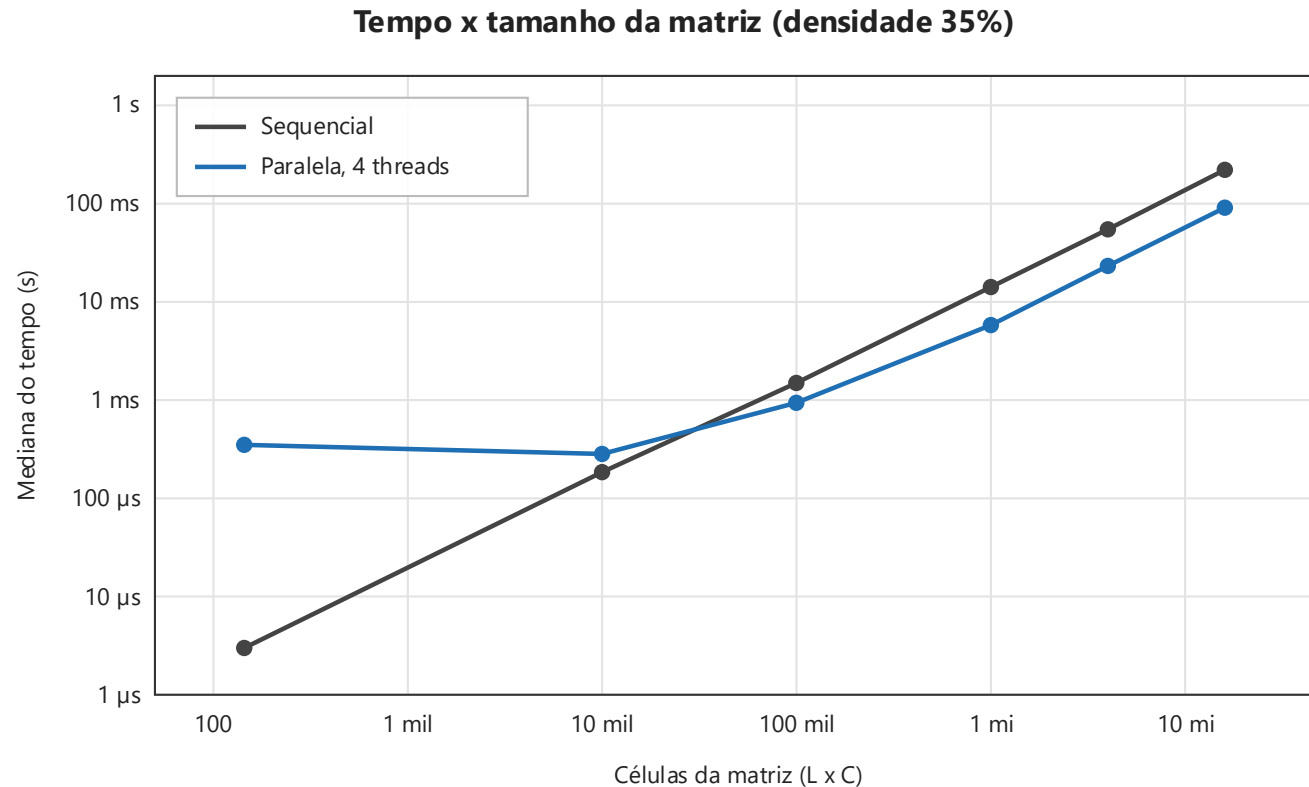
```
for (i = 0; i <= total; i++) pai[i] = i; /* 16 mi */
/* ... threads, join, fronteiras ... */
for (i = 0; i < total; i++) { /* 16 mi */
    if (rotulos[i] == 0) continue;
    r = uf_find(pai, rotulos[i]);
    if (!visto[r]) { visto[r] = 1; objetos++; }
}
```

Agora

```
/* em cada thread, ao criar o rotulo */
pai[idx + 1] = idx + 1;

/* thread principal, depois do join */
for (i = 0; i < nthreads; i++)
    objetos += trab[i].componentes;
objetos -= consolida_frenteiras(...);
```

# Quando o paralelo compensa



- $12 \times 12$ : sequencial **0,003 ms**, paralelo **0,351 ms**: mais de 100× mais lento.
- Criar e esperar as threads custa **cerca de 0,3 ms**, mais que contar 144 células.
- Empate entre **10 mil e 100 mil** células.
- A partir de **1 milhão** de células:  $S \approx 2,4$  com 4 threads.

Nas matrizes do enunciado o paralelo é mais lento, e isso é esperado: elas servem para validar a **correção**, não o desempenho.

# Conclusões e demonstração

## Correção

Sequencial e paralelo dão o mesmo resultado em todas as matrizes e quantidades de threads.

## Paralelismo real

Cada thread rotula sua faixa; sem mutex, porque as escritas são disjuntas.

## Fronteiras

Union-Find e a conta  $\Sigma$  locais – uniões resolvem os objetos que cruzam as faixas.

## Desempenho

Até 4,5× com 16 threads na matriz grande; nas pequenas, o custo das threads domina.

### Demonstração

```
make && make test           # compila e roda as 5 obrigatorias
./bin/paralelo 4 tests/obrigatorios/ex2.txt # objeto cruzando a divisao
./bin/paralelo 8 -g 4000 4000 35 2026      # matriz grande
make help                        # todas as opcoes
```